

Q1. Arithmetic Expression Evaluation Using YACC

(i) YACC Grammar

```
%{  
#include <stdio.h>  
%}  
%token NUMBER  
%left '+'  
%left '*'  
%%  
E : E '+' E { $$ = $1 + $3; }  
  | E '*' E { $$ = $1 * $3; }  
  | NUMBER { $$ = $1; }  
;  
%%
```

(ii) Semantic Actions

- \$1 = first operand
- \$3 = second operand
- \$\$ = calculated result

$E + E \rightarrow \$\$ = \$1 + \$3$

$E * E \rightarrow \$\$ = \$1 * \$3$

(iii) Evaluate $3 + 4 * 5$

Since $*$ has higher precedence:

$4 * 5 = 20$

$3 + 20 = 23$

(iv) Output

Input : $3 + 4 * 5$

Result : 23

Conclusion: YACC evaluates the expression correctly using precedence and semantic actions.

Q2. Syntax Tree / AST Using YACC

(i) Grammar

%token ID

%%

E : E '+' E

{ \$\$ = createNode("+",\$1,\$3); }

| E '*' E

{ \$\$ = createNode("*",\$1,\$3); }

| ID

{ \$\$ = createNode(\$1,NULL,NULL); }

;

%%

(ii) Semantic Action

Operators are internal nodes and identifiers are leaf nodes.

a + b * c

(iii) AST

+

/ \

a *

/ \

b c

(iv) Traversals

Preorder : + a * b c

Inorder : a + b * c

Postorder: a b c * +

Conclusion: The AST represents operator precedence and can be used for semantic analysis and code generation.

Q3. Synthesized Attributes in YACC

(i) SDD using \$1, \$3, \$\$

E : E '+' T { \$\$ = \$1 + \$3; }

| T { \$\$ = \$1; }

;

T : T '*' F { \$\$ = \$1 * \$3; }

```

| F      { $$ = $1; }
;
F : NUMBER { $$ = $1; }
;

```

Here, \$\$ is the synthesized attribute of the left-hand side.

(ii) Evaluate $2 * 3 + 4$

$$2 * 3 = 6$$

$$6 + 4 = 10$$

Result = 10

(iii) Attribute Propagation

$$2 \rightarrow F \rightarrow T = 2$$

$$3 \rightarrow F = 3$$

$$T * F \rightarrow 2 * 3 = 6$$

$$4 \rightarrow F \rightarrow T = 4$$

$$E + T \rightarrow 6 + 4 = 10$$

(iv) Bottom-Up Evaluation

Tokens $\rightarrow$ Reductions $\rightarrow$ Attribute Values $\rightarrow$ Final Result

$$2,3,4 \rightarrow F,T,E \rightarrow 2,6,10 \rightarrow 10$$

Conclusion: YACC performs synthesized-attribute evaluation naturally during bottom-up parsing.

Q3. Synthesized Attributes in YACC

(i) SDD using \$1, \$3, \$\$

```

E : E '+' T { $$ = $1 + $3; }
| T      { $$ = $1; }
;
T : T '*' F { $$ = $1 * $3; }
| F      { $$ = $1; }
;
F : NUMBER { $$ = $1; }
;

```

Here, \$\$ is the synthesized attribute of the left-hand side.

(ii) Evaluate $2 * 3 + 4$

$$2 * 3 = 6$$

$$6 + 4 = 10$$

Result = 10

(iii) Attribute Propagation

$$2 \rightarrow F \rightarrow T = 2$$

$$3 \rightarrow F = 3$$

$$T * F \rightarrow 2 * 3 = 6$$

$$4 \rightarrow F \rightarrow T = 4$$

$$E + T \rightarrow 6 + 4 = 10$$

(iv) Bottom-Up Evaluation

Tokens $\rightarrow$ Reductions $\rightarrow$ Attribute Values $\rightarrow$ Final Result

$$2, 3, 4 \rightarrow F, T, E \rightarrow 2, 6, 10 \rightarrow 10$$

Conclusion: YACC performs synthesized-attribute evaluation naturally during bottom-up parsing.

Q5. Type Coercion in YACC

(i) Grammar

declaration

: FLOAT ID ';' { declare(\$2, "float"); }

| INT ID ';' { declare(\$2, "int"); }

;

assignment

: ID '=' expression ';' ;

;

expression

: ID

| INTEGER

| REAL

;

(ii) Given Program

float temperature;

```
int sensor_value;  
temperature = sensor_value;
```

(iii) Symbol Table

Variable	Type
----------	------

temperature	float
-------------	-------

sensor_value	int
--------------	-----

(iv) Implicit Conversion

The compiler checks:

LHS = float

RHS = int

So it performs:

$\text{int} \rightarrow \text{float}$

For example:

```
sensor_value = 25
```

becomes:

```
temperature = 25.0
```

Conceptually:

```
temperature = (float)sensor_value;
```

(v) Final Result

temperature : float

sensor_value : int

Conversion : $\text{int} \rightarrow \text{float}$

Assignment : Compatible

Final type of temperature : float

Conclusion: YACC semantic actions and the symbol table allow the compiler to perform automatic type coercion from int to float.